

Pandas 数据处理

完整学习讲义

清华大学统计与数据科学系 俞声 2026.5.27

第 1 章 为什么使用 Pandas?

1.1 表格数据的重要性

在数据科学中，**表格数据 (Tabular Data)** 是最常见的数据形式。每一行代表一个观测样本，每一列代表样本的某种属性（特征）。

典型问题示例（以教授工资数据集为例）

- 不同职称的平均工资是多少?
- 工资与工龄之间有什么关系?
- 性别、学科对工资的影响是否显著?

1.2 为何不用原生 Python / NumPy?

工具	局限性
原生 Python (列表 / 字典)	功能不足，缺乏高效的表格操作；速度较慢
NumPy	矩阵只能存单一类型；缺乏数据分析所需的高级功能
Pandas	专为表格数据分析设计；底层使用 NumPy，兼顾速度与功能

Pandas 在 NumPy 之上构建，既保留了数组运算的高性能，又增加了适合数据分析的丰富 API。

第 2 章 核心数据结构

2.1 Series — 一维带标签数组

Series 是 Pandas 中最基本的数据结构，是一个**一维、带标签的数组**。它包含：

- **值 (values)**：同一类型的数据序列
- **索引 (index)**：每个值对应的标签，默认为 0, 1, 2, ...

创建 Series

```
import pandas as pd

# 默认整数索引

s = pd.Series([10, 25, 100])

# s.index → RangeIndex(start=0, stop=3, step=1)
# s.values → array([ 10,  25, 100])

# 自定义字符串索引

s = pd.Series([10, 25, 100], index=["枪兵", "狮鹫", "骑兵"])

# 修改索引

s.index = ["Pikeman", "Griffin", "Cavalier"]
```

Series 的选取

可通过三种方式从 Series 中取值：

方式	语法示例	说明
单标签	s['枪兵']	返回对应的单个值
标签列表	s[['骑兵', '狮鹫']]	返回子 Series
布尔条件	s[s > 20]	返回满足条件的子 Series

```
s = pd.Series([10, 25, 100], index=["枪兵", "狮鹫", "骑兵"])

s["枪兵"]          # → 10

s[['骑兵', "狮鹫"]] # → Series(骑兵:100, 狮鹫:25)
```

```
s > 20                                # → Series(枪兵:False, 狮鹫:True, 骑兵:True)

s[s > 20]                             # → Series(狮鹫:25, 骑兵:100)
```

2.2 DataFrame — 二维表格

DataFrame 是 Pandas 的核心对象，可以看作是多个等长 Series 共享同一个 Index 组成的集合。

- 每一列是一个 Series（具有相同 Index）
- 列名（column labels）通常代表特征名
- 行标签（Index）代表每条观测

记忆要点

DataFrame = 若干列 Series 的有序集合

Series = 带标签的一维数组

Index = 行标签，不一定唯一，不一定是数字

第 3 章 数据的读取、创建与写出

3.1 从 CSV 文件读取

```
import pandas as pd

data = pd.read_csv("HoMM3Units.csv")
```

pd.read_csv 常用参数

参数	作用	示例
sep / delimiter	列分隔符（默认逗号）	sep='t'（制表符分隔）
encoding	文件编码	encoding='utf-8' 或 'gb18030'
header	哪一行作为列名	header=0（第 1 行）， header=None（无列名）
skiprows	跳过开头若干行	skiprows=2 或 skiprows=[0,2,4]
nrows	最多读取行数	nrows=10000（只读前 1 万行）
index_col	指定行索引列	index_col=0 或 index_col='id'

非数值列被当作 object 类型的处理

Pandas 继承 NumPy 的局限性, 非数值列默认存为 object, 效率低。有两种解决方式:

```
# 方式 1: 读取后转换 (简单但需额外时间)

pd.read_csv("HoMM3Units.csv").convert_dtypes()

# 方式 2: 直接使用 pyarrow 后端 (效率更高)

pd.read_csv("HoMM3Units.csv", dtype_backend="pyarrow")
```

3.2 从 Series / 字典创建 DataFrame

```
s_names = pd.Series(["Pikeman", "Griffin", "Cavalier"])

s_health = pd.Series([10, 25, 100])

# 用字典把多个 Series 打包成 DataFrame

data = pd.DataFrame({"Unit_name": s_names, "Health": s_health})
```

3.3 从 JSON (JSONL) 读写

```
# 写出: 每行一个 JSON 对象 (JSONL 格式)

data.to_json("HoMM3Units.jsonl", orient="records", lines=True,
force_ascii=False)

#orient="records": table 里的每一行为一个单位

# 读取 JSONL

data = pd.read_json("HoMM3Units.jsonl", lines=True)
```

其他创建方式: 列表、列表的列表、字典的列表、列表的字典——Pandas 几乎可以从任何常见的 Python 数据结构创建 DataFrame。

3.4 写出到 CSV

```
df.to_csv(

    "output.csv",          # 文件路径

    sep="," ,              # 分隔符 (默认逗号)

    encoding="utf-8",      # 编码 (中文推荐 utf-8 或 gbk)
```

```
index=False,          # 不写行索引（默认会多出一列）

header=True,          # 写入列名（默认 True）

columns=["name","age"], # 只写指定列

na_rep="NULL",        # 缺失值用 NULL 表示

quotechar='"',        # 引号字符

)
```

第 4 章 索引 (Index) 系统

4.1 什么是 Index?

Index 即行标签，它有以下特性：

- 默认为整数序列 (0, 1, 2, ...)
- 可以是字符串、日期等任意类型
- 可以有名字（用 `index.name` 设置）
- 不要求唯一，但列名（column names）通常应唯一

4.2 设置与重置 Index

```
# 读取时指定 Index 列

data = pd.read_csv("HoMM3Units.csv", index_col="Unit_name")

# 读取后设置 Index（修改原 DataFrame）

data.set_index("Unit_name", inplace=True)

# 重置回默认整数 Index

data.reset_index(inplace=True)
```

。inplace 参数说明

大多数 DataFrame 操作默认返回新 DataFrame，不修改原对象。

加上 `inplace=True` 才会就地修改，函数返回 `None`。

建议：新手先不用 `inplace`，写 `data = data.xxx(...)`，更直观。

4.3 获取行列信息

<code>data.index</code>	# 行标签 (Index 对象)
<code>data.columns</code>	# 列名 (Index 对象)
<code>data.index.tolist()</code>	# 转换为 Python 列表
<code>data.columns.tolist()</code>	
<code>data.shape</code>	# (行数, 列数)

第 5 章 数据提取

5.1 快速查看: head 与 tail

<code>df.head(5)</code>	# 前 5 行 (默认)
<code>df.head(n)</code>	# 前 n 行
<code>df.tail(5)</code>	# 最后 5 行
<code>df.tail(n)</code>	# 最后 n 行

5.2 基于标签的提取: .loc

.loc 使用行/列的 **标签名** 来定位数据。第一个参数是行, 第二个参数是列。

.loc 的参数类型

- 单个标签 → 返回单值或 Series
- 标签列表 → 返回子 DataFrame
- 切片 `a:b` → 包含两端 (与 Python 切片不同!)
- 冒号: → 选取全部
- 布尔数组 → 选取 True 对应的行/列

用列表选行和列
<code>data.loc[[1, 5, 9], ["Unit_name", "Attack", "Defence"]]</code>
用切片选行和列 (注意: 两端都包含!)
<code>data.loc[0:5, "Unit_name":"Defence"]</code>

```
# 选所有行, 指定列
data.loc[:, ["Unit_name", "Attack", "Defence"]]

# 选所有列, 指定行切片
data.loc[0:5, :]

# 单值提取 (返回基础 Python 类型)
data.loc[0, "Unit_name"]    # → 退化为"Pikeman"
data.loc[[0], ["Unit_name"]] # → 单元素 df

# 切片提取 (返回 Series)
data.loc[0:5, "Unit_name"]  # → 退化为 Series
data.loc[0:5, ["Unit_name"]] # → 单列的 df
```

5.3 基于整数位置的提取: .iloc

.iloc 使用行/列的 **整数位置 (0 起始)** 来定位数据。

iloc 切片不包含右端 (与 Python 切片行为相同!)

.loc[0:5] → 包含第 0、1、2、3、4、5 行 (6 行)

.iloc[0:5] → 包含第 0、1、2、3、4 行 (5 行, 不含第 5 行)

```
# 列表选取
data.iloc[[1, 5, 9], [0, 3, 4]]

# 切片选取 (右端不包含)
data.iloc[0:6, 0:5]    # 行 0-5, 列 0-4

# 单值
data.iloc[0, 0]        # → "Pikeman"

# 单列切片
data.iloc[0:6, 0]      # → Series
```

5.4 .loc vs .iloc 对比

	.loc	.iloc
定位依据	标签 (label)	整数位置 (integer position)
切片右端	包含 (inclusive)	不包含 (exclusive)
适用场景	大多数情况首选	需要按位置取数时
可读性	更高 (知道列名含义)	较低 (只有数字)
安全性	数据顺序变化后仍正确	数据重排后可能错误

5.5 上下文敏感提取：[]

方括号 [] 只接受一个参数，根据参数类型自动决定行为：

参数类型	行为	示例
行整数切片	按位置提取行	data[3:7]
列标签列表	提取多列 → DataFrame	data[['Unit_name','Attack']]
单列标签	提取单列 → Series	data['Unit_name']

data[3:7]	# 第3~6行 (切片行为)
data[['Unit_name', 'Attack', 'Defence']]	# 多列
data['Unit_name']	# 单列 → Series

5.6 条件筛选（布尔索引）

.loc 和 [] 均支持布尔数组作为参数，True 对应的行被保留，False 的行被过滤。

Health > 100 的行
data[data['Health'] > 100]
阵营为 Conflux
data.loc[data['Faction'] == 'Conflux']
7 级以上且有飞行能力 (组合条件)

```
data[(data['Level'] >= '7') & data['Special_abilities'].str.contains('Flying')]
```

注意: 组合条件用 & (与)、| (或)、~ (非)

不能用 and / or / not

每个条件必须用括号括起来!

组合条件的运算符

& → 逻辑"与" 例: (A) & (B)

| → 逻辑"或" 例: (A) | (B)

~ → 逻辑"非" 例: ~(A)

禁止用 and / or / not (它们作用于整个 Series 会出错)

第 6 章 列操作

6.1 新增列

通过方括号 [] 给新列名赋值, 值为等长的 Series 或数组:

新增布尔列: 不含 'Undead' 关键词 → 是生物单位

```
data['Is_living'] = ~data['Special_abilities'].str.contains('Undead')
```

6.2 修改列

向已有列名赋新值即可修改:

修改 Is_living: 还要排除 Unliving

```
data['Is_living'] = data['Is_living'] &
~data['Special_abilities'].str.contains('Unliving')
```

6.3 重命名列

rename 接受字典: {旧列名: 新列名}

```
data.rename(columns={'Is_living': 'Living'}, inplace=True)
```

6.4 删除列 (或行)

```
# 删除列 (axis="columns" 表示操作列)

data.drop('Living', axis='columns', inplace=True)

# 删除行 (axis="rows" 或 axis=0, 默认)

data.drop([0, 1, 2], axis="rows", inplace=True)
```

常见陷阱

`drop()` 默认操作的是行 (`axis=0`)，删列必须指定 `axis="columns"` (或 `axis=1`)。不加 `inplace=True` 时，`drop` 返回新 `DataFrame`，原数据不变。

第 7 章 常用工具函数

7.1 数学运算 (NumPy 集成)

```
import numpy as np

np.mean(data['Speed'])          # 平均速度

np.max(data['Gold'])            # 最高金费

data.loc[np.argmax(data['Gold']), 'Unit_name'] # 最贵单位名
```

7.2 shape 与 size

```
data.shape # → (141, 14) 行数 × 列数

data.size  # → 1974      总元素数 = 141 × 14
```

7.3 describe — 统计摘要

`describe()` 对数值列计算: `count`、`mean`、`std`、`min`、25%/50%/75% 分位数、`max`。

```
data.describe()          # 数值列的统计摘要

data['Speed'].describe() # 单列
```

7.4 sample — 随机抽样

```
data.sample()          # 随机抽 1 行
data.sample(4)          # 随机抽 4 行 (不放回)
data.sample(40, replace=True) # 有放回地抽 40 行

# 设置随机种子保证可复现【涉及抽样一定要锁种子!】
np.random.seed(100)     # 影响 NumPy 和 Pandas
data.sample(10)
```

7.5 value_counts — 频率统计

`value_counts()` 统计 Series 中每个唯一值出现的次数, 结果按频率降序排列。

```
data['Faction'].value_counts()          # 各阵营单位数量
data['Faction'].sample(100).value_counts() # 抽样后统计
```

7.6 unique — 去重枚举

```
data['Faction'].unique() # 返回所有唯一阵营名 (NumPy 数组)
```

7.7 sort_values — 排序

```
# Series 排序
data['Unit_name'].sort_values()

# DataFrame 按列排序
data.sort_values('AI_value', ascending=False) # 降序
data.sort_values('Speed', ascending=True)     # 升序 (默认)
```

第 8 章 分组聚合 (GroupBy)

8.1 为什么要分组?

当需要对不同类别的行分别执行相同操作时, 分组提供了比循环更简洁、高效的语法。

例: 计算每个阵营 (Faction) 的平均攻击力、防御力、生命值。

8.2 分组三步曲：Split → Apply → Combine

步骤	含义	操作
Split（分组）	按某列值将 DataFrame 拆为若干子 DataFrame	groupby("列名")
Apply（应用）	对每个子 DataFrame 应用函数	.apply(func)
Combine（合并）	将各子结果合并为最终结果	自动完成

8.3 groupby 基础用法

```
# 生成 DataFrameGroupBy 对象
groups = data.groupby('Faction')

# 每个 group 是一个"迷你 DataFrame"
# 包含该阵营（Faction）的所有行
```

8.4 .apply() — 聚合（返回标量）

当传入函数对每个子 DataFrame 返回一个标量时，称为聚合（Aggregation）：

```
# 每个阵营的总战力 = AI_value × Growth 的点积
data.groupby('Faction').apply(
    lambda df: df['AI_value'] @ df['Growth']
)
```

8.5 .apply() — 变换（返回等长 Series）

当传入函数对每个子 DataFrame 返回与子 DataFrame 等长的 Series 时，称为变换（Transformation）：

```
# 计算每个单位在其阵营中的战力百分比
data['Power%'] = data.groupby('Faction', group_keys=False).apply(
    lambda df: df['AI_value'] * df['Growth'] / (df['AI_value'] @ df['Growth'])
)

# group_keys=False 防止生成 MultiIndex
```

【也是 map-reduce，相同函数放到各个行，加速】

8.6 内置聚合函数（更高效）

当只涉及单列操作时，可以跳过 `.apply()` 直接使用内置方法：

```
# 每个阵营速度最慢的单位 (idxmin 返回最小值所在的 Index)
data.loc[data.groupby('Faction')['Speed'].idxmin(),
         ['Faction', 'Unit_name', 'Speed']]

# 其他常用内置: .mean() .sum() .max() .min() .count() .std()
```

g 参考: https://wesmckinney.com/book/data-aggregation#groupby_aggregation

第 9 章 合并与拼接

9.1 merge — 基于列的连接

类似 SQL 的 JOIN，`pd.merge()` 根据一个或多个共同列的键值合并两个 DataFrame。

四种连接类型

类型	参数	保留行	说明
Left Join	<code>how='left'</code>	左表全部行	最常用！
Right Join	<code>how='right'</code>	右表全部行	较少用
Inner Join	<code>how='inner'</code>	两表共同键的行	默认值（注意！）
Outer Join	<code>how='outer'</code>	两表所有行（并集）	可能产生大量 NaN

重要提醒：默认是 Inner Join！

`pd.merge(A, B, on='x1')` 默认 `how='inner'`，会丢弃不匹配的行。
实际工作中 Left Join 更安全，记得显式写 `how='left'`。

```
# 基本用法

pd.merge(A, B, on='x1', how='left')    # 左连接

pd.merge(A, B, on='x1', how='inner')    # 内连接（默认）
```

```
# 左右键名不同时
```

```
pd.merge(A, B, left_on='x1', right_on='X1', how='left')
```

```
# 用 Index 作为连接键
```

```
pd.merge(A, B, left_on='x1', right_index=True, how='left')
```

9.2 concat — 拼接

`pd.concat()` 将多个 DataFrame 按行或列方向"堆叠"拼接:

```
A = pd.DataFrame({"x1": ["A", "B", "C"], "x2": [1, 2, 3]}, index=[1, 2, 3])
```

```
B = pd.DataFrame({"X1": ["B", "C", "D"], "X2": [2, 3, 4]})
```

```
# 按列拼接 (不对齐行 Index)
```

```
pd.concat([A, B], axis="columns")
```

```
# 按列拼接 (重置行 Index 再拼)
```

```
pd.concat([A.reset_index(drop=True), B.reset_index(drop=True)], axis="columns")
```

```
# 按行拼接 (列名不统一, 会出现 NaN)
```

```
pd.concat([A, B], axis="rows", ignore_index=True)
```

```
# 按行拼接 (先统一列名)
```

```
B.columns = A.columns
```

```
pd.concat([A, B], axis="rows", ignore_index=True)
```

9.3 join — 基于 Index 的连接

`.join()` 是 `merge` 的简化版, 默认按 Index 连接 (类似 Left Join):

```
# 将翻译表连接到原数据 (按 Unit_name 连接)
```

```
data_translate.set_index("EN", inplace=True)
```

```
data_cn = data.join(data_translate, on="Unit_name")
```

9.4 map / replace — 值替换

对于简单的值映射（如将英文名翻译成中文），map 和 replace 更简洁：

```
# 构建映射字典
en_to_cn = dict(zip(data_translate["EN"], data_translate["CN"]))

# map: 对 Series 的每个值查字典
data['Unit_name'] = data['Unit_name'].map(en_to_cn)

# replace: 可同时对多列批量替换
data[["Unit_name", "Faction"]] = data[["Unit_name",
"Faction"]].replace(en_to_cn)
```

第 10 章 总结与学习建议

10.1 知识图谱速览

主题	核心方法 / 概念
数据结构	Series, DataFrame, Index
读写数据	read_csv, to_csv, read_json, to_json
标签提取	.loc[行标签, 列标签]
位置提取	.iloc[行号, 列号]
简洁提取	[] 切片/列表/单列
条件筛选	bool 数组 + &, , ~
列操作	赋值新列, rename, drop
统计函数	describe, value_counts, sort_values, sample
分组聚合	groupby + apply / agg / transform
合并拼接	merge (left/inner/right/outer), concat, join, map

10.2 常见错误汇总

错误类型	原因	解决方法
Index 不匹配	合并或赋值时行索引不对齐	先 <code>reset_index()</code> 或用 <code>.values</code>
数据类型不匹配	字符串与数字比较/合并	用 <code>convert_dtypes()</code> 或手动转型
&/ 忘加括号	运算符优先级问题	每个条件加括号：(A) & (B)
inplace 误用	以为就地修改了，实则没有	用返回值： <code>df = df.drop(...)</code>
.loc 切片含右端	忘记与 .iloc 的区别	<code>.loc[0:5]</code> 含第 5 行！
drop 忘写 axis	默认按行删，列删失败	<code>axis='columns'</code>

10.3 学习建议

- 动手实践最重要——用真实数据集练习比死记 API 有效 10 倍
- **先掌握"增删改查"**：`.loc`, `.iloc`, `[]`, 条件筛选, 列操作
- 遇到不熟悉的函数，先查官方文档：pandas.pydata.org
- `groupby` 是难点，多动手模拟 Split-Apply-Combine 三步
- 推荐参考书：《Python for Data Analysis》（Wes McKinney 著）

最后一句话

"Pandas can be a bit tricky to master at first — Keep practicing!"

（Pandas 初学时确实有点绕，但坚持练就会越来越顺手！）